

Arquivo: local_storage.py

```
"""
DESCRIÇÃO GERAL:
Esta é a camada de persistência local de cada nó.
Ela manipula arquivos diretamente no disco e pode ser reaproveitada por qualquer
nó do cluster, apenas mudando a raiz física onde os dados são salvos.
"""

from pathlib import Path

from dfs.config import STORAGE_DIR

class LocalStorage:
    """
    Implementa o armazenamento local de um nó.
    """

    def __init__(self, root: Path | None = None):
        # Se nenhum diretório for informado, usa a raiz padrão.
        self.root = Path(root) if root is not None else STORAGE_DIR

        # Garante que a pasta de armazenamento exista.
        self.root.mkdir(parents=True, exist_ok=True)

    def _resolve_path(self, path: str) -> Path:
        """
        Resolve o caminho final e protege contra path traversal simples.
        """
        # Resolve o caminho completo do arquivo.
        target = (self.root / path).resolve()

        # Garante que o caminho final continue dentro da raiz do storage.
        root_resolved = self.root.resolve()
        if root_resolved not in target.parents and target != root_resolved:
            raise ValueError("Caminho inválido fora da raiz do storage")

        return target

    def put(self, path: str, data: bytes) -> None:
        """
        Salva um arquivo em disco.
        """
        target = self._resolve_path(path)

        # Cria diretórios intermediários se necessário.
        target.parent.mkdir(parents=True, exist_ok=True)

        # Escreve os bytes no arquivo.
        target.write_bytes(data)

    def get(self, path: str) -> bytes:
```

```
"""
Lê um arquivo do armazenamento local.
"""
target = self._resolve_path(path)
return target.read_bytes()

def delete(self, path: str) -> None:
    """
    Remove um arquivo do disco.
    """
    target = self._resolve_path(path)
    target.unlink()

def list_files(self) -> list[str]:
    """
    Lista todos os arquivos armazenados de forma recursiva.
    """
    return sorted(
        p.relative_to(self.root).as_posix()
        for p in self.root.rglob("*")
        if p.is_file()
    )
```